

# Operációs rendszerek II.

## UNIX: folyamatok ütemezése, call-out mechanizmus



Várkonyiné Kóczy Annamária

Professzor

Informatika Tanszék

[koczya@uj.ssk](mailto:koczya@uj.ssk)

[varkonyi-koczy@uni-obuda.hu](mailto:varkonyi-koczy@uni-obuda.hu)

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



# 14. Folyamatok ütemezése

- Bevezetés
- A tradicionális UNIX ütemezés
- Ütemezési mintapéldák
- Modern UNIX ütemezők
- Call-out mechanizmus
- Megvalósítás láncolt listával
- Megvalósítás időkerékkel

# Bevezetés (ism.)

- Ütemezés:
  - kiválasztani a futásra kész folyamatok közül a következő futót
  - a rendszer teljesítménye és HW kihasználtsága szempontjából a legfontosabb
- Alapfogalmak
  - preemptív illetve ill. nem preemptív (kooperatív) ütemező
  - mértékek: CPU kihasználtság, átbecsátó képesség, várakozási idő, körülfordulási idő, válaszüő
  - prioritás
  - statikus és dinamikus ütemezés
- A működés alapelve
  - a feladatokat sorokba (task/job queue) rendezzük
  - az ütemező a sorokból választja ki a következő futó folyamatot

# Bevezetés (folyt.)

- Követelmények (általános)
  - minél kisebb erőforrásigény (overhead), komplexitás
  - optimalitás a mértékek valamilyen kombinációja szerint
  - determinisztikus (jósolható viselkedés)
  - fair
  - elkerüli a kiéhezést
  - összeomlás elkerülése (graceful degradation)
  - speciális körülmények egyedi igényei (real-time, batch, multimédia, stb.)

# A tradicionális UNIX ütemezőkkel szembeni elvárások

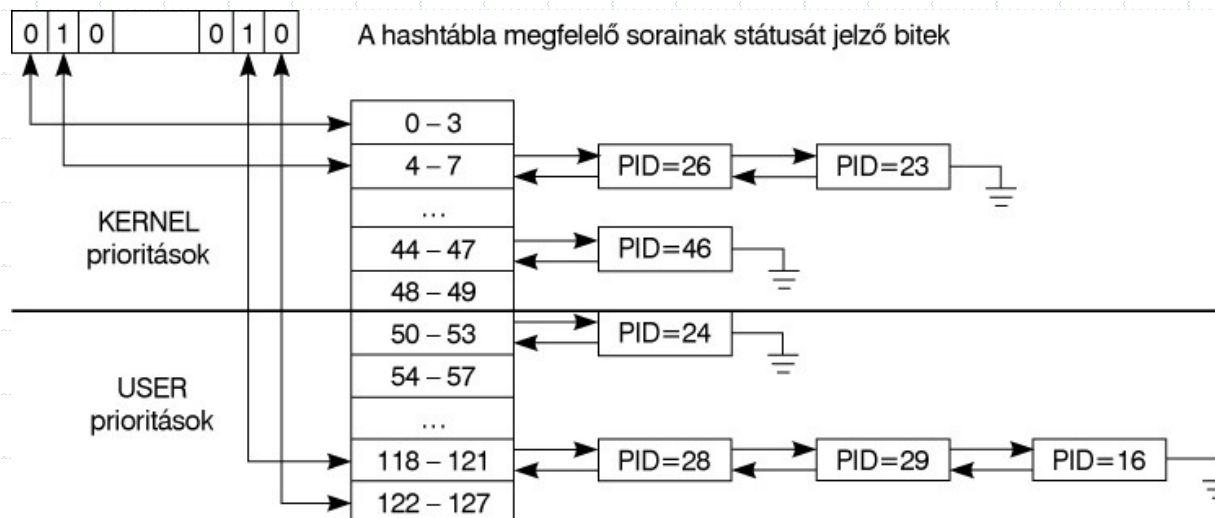
- több felhasználós, interaktív és batch folyamatokat futtató felhasználói környezet
  - Alacsony válaszidő
  - Nagy átbocsátó képesség
  - Alacsony prioritású, háttérben futó folyamatok éheztetésének elkerülése
- + általános kritériumok
  - Pl. graceful degradation
  - A felhasználó befolyásolhassa a folyamatok futási esélyeit

# A tradicionális UNIX ütemezők jellemzése

- A UNIX ütemező preemptív, prioritásos és időosztásos
- Az ütemezés **prioritásos**
  - minden folyamat kap egy dinamikusan számított prioritás értéket
  - a legnagyobb prioritású folyamatot választja a futásra kész sorból
- Az ütemezés **időosztásos**
  - több folyamat konkurens futását támogatja
  - minden folyamat kap egy időszeletet, amíg futhat
  - az idő letelte után az azonos prioritásúak közül a következő fog futni
- **A felhasználói és kernel mód ütemezése eltérő**
  - felhasználói módban: preemptív és időosztásos, időben változó prioritás, azonos prioritásnál RR vagy FCFS
  - kernel módban: nem preemptív, nincs időosztás, rögzített prioritás

# Az ütemezés adatstruktúrái

- A prioritás 0-127 közötti értéket vehet fel
  - a 0 a legmagasabb prioritás, a 127 a legkisebb
  - 0-49: kernel prioritási szintek 50-127: felhasználói szintek
- Prioritások tárolása
  - Dinamikus adatszerkezet , láncolt listás, listafejek egy hash táblában, 1 fej – 4 prioritás
  - Előnyök: kisebb hashtábla, gyors ellenőrzés, éppen 32 bit



# Ütemezés kernel módban

- A kernel módú ütemezés tulajdonságai:
  - a prioritás statikus
  - nem preemptív, így egyszerűbb a kernel adatstruktúrák védelme
- A prioritás nem függ attól, hogy
  - mennyi volt a prioritása felhasználói módban
  - mennyi processzor időt kapott korábban
- A kernel módú prioritás meghatározása
  - A prioritást a folyamat elalvásának az oka határozza meg, ütemezéskor határozza meg.
  - Ez az ún. **alvási prioritás** (sleep priority).
  - A folyamat felébredése után ez fogja meghatározni a prioritását.
  - Az alvási prioritás az alvás okához kötött, pl.:
    - 20: diszk I/O-ra vár
    - 28: inputra vár a karakteres terminálról



# Ütemezés kernel módban

- Mikor fut egy folyamat rendszermódban?
- TRAP: itt ki tudja számolni a prioritást a TRAP alapján
- külső IT: nem tudja kiszámolni, de nem is kell, nincs újraütemezés!
  - Folyamatkörnyezetben hajtódik végre (az idő is a folyamat futásához számolódik), utána visszatér oda, ahonnan jött

# Ütemezés felhasználói módban

- Felhasználói módban az ütemezés alapja a CPU használat (számított prioritás)
- A prioritás némileg befolyásolható egy külső prioritás-elemmel, amit a felhasználótól kap (*nice*)
- Ütemezés
  - mindig a legnagyobb prioritású folyamatot ütemezi be a kernel
  - megnézi a hash táblát, és kiválasztja a legmagasabb futási szintet, ahol van folyamat
  - ha ez magasabb, mint a futó folyamat szintje, akkor átütemez
  - ha ez azonos a futó folyamat szintjével, és letelt a futó folyamat időszelete, akkor szintén átütemez

# Ütemezés felhasználói módban (folyt.)

- Az ütemezési tevékenység idő szerint rendezve
  - minden óraciklusban
    - Van-e magasabb szinten folyamat? Ha igen, akkor átütemezés.
    - A legmagasabb, folyamatot tartalmazó futási szint sorban első folyamata fog futni.
  - minden időszeglet végén (10 óraciklus)
    - Van-e másik folyamat az éppen futó folyamat prioritásának megfelelő futási szinten? Ha igen, akkor átütemezés.
    - Az adott szint első folyamata fog futni. A preemptált folyamat a sor végére kerül, azaz az időosztásos ütemezési rész Round-Robin algoritmus szerinti.
  - Minden 100. óraciklusban
    - Prioritás újraszámolás, ennek megfelelő újraütemezés

# Az ütemezés további adatai (prioritás számításhoz)

- A kernel a következő adatokkal írja le a folyamatokat
  - p\_pri            a folyamat aktuális prioritása
  - p\_usrpri        a folyamat felhasználói módban érvényes prioritása
  - p\_cpu           a CPU használat mértékére vonatkozó szám
  - p\_nice          a felhasználó által adott prioritás módosító értéke
- Az ütemező aktuális prioritás érték alapján dönt az átütemezésről
- Kernel módba váltáskor a prioritás értéke „elmentődik” annak érdekében, hogy felhasználói módba visszalépéskor visszaállítható legyen a p\_usrpri értékéből.
- A CPU érték kifejezi, hogy a folyamat az elmúlt időszakban mennyi processzor időt kapott. Ez csökkenteni fogja a prioritását, azaz biztosítható a fair ütemezés.

# A prioritás számítása felhasználói módban

- Minden óraciklusban a futó folyamatra növeli a `p_cpu` értékét

`p_cpu++`

- Minden 100. óraciklusban a kernel kiszámítja a prioritás értékét

- minden folyamatnál „öregíti” a `p_cpu` értéket

`p_cpu = p_cpu * KF` (KF: korrekciós faktor)

- kiszámolja a prioritást

`p_pri = P_USER + p_cpu / 4 + 2 * p_nice`

(`P_USER`: konstans, = 50, hogy a rendszermódú foly.k ...)

- A korrekciós faktor számítása

- SVR3: `KF = 1/2`

- 4.3 BSD: a futásra kész folyamatok átlagos számától (`load_avg`) függ: `KF = 2 * load_avg / (2 * load_avg + 1)`

# A felhasználói módú ütemezés összefoglalója

- Minden óraütésnél
  - Ha van magasabb prioritási sorban folyamat, akkor átütemezés
  - A futó folyamat  $p\_cpu$  értékének növelése (csökkenni fog a prioritása):  $p\_cpu := p\_cpu + 1$
- Minden 10. óraütésnél
  - Round-Robin ütemezés a legmagasabb (felhasználói) futási szinten levő folyamatok között
- Minden 100. óraütésnél
  - a korrekciós faktor kiszámítása az elmúlt 100 ciklus átlagos terhelése alapján, pl.:  $KF = 2 * n / (2 * n + 1)$
  - A CPU használat ( $p\_cpu$ ) „öregítése”:  $p\_cpu = p\_cpu * KF$
  - a prioritás újraszámolása minden folyamatra:  
$$p\_pri = P\_USER + p\_cpu / 4 + 2 * p\_nice$$
  - szükség szerint átütemezés

# Ütemezési mintapéldák

- Három folyamat, Round-Robin ütemezés nélkül

| A                 |             | B                 |             | C     |       |       |               |
|-------------------|-------------|-------------------|-------------|-------|-------|-------|---------------|
| p_pri             | p_cpu       | p_pri             | p_cpu       | p_pri | p_cpu | Lépés | Futó folyamat |
| 50                | 0           | 50                | 0           | 50    | 0     | 1     | A             |
| 50                | 1           | 50                | 0           | 50    | 0     | 2     | A             |
| ...               | ...         | ...               | ...         | ...   | ...   | ...   | A             |
| 50                | 99          | 50                | 0           | 50    | 0     | 99    | A             |
| 50+50/<br>4<br>63 | 100/2<br>50 | 50                | 0           | 50    | 0     | 100   | A             |
| 63                | 50          | 50                | 1           | 50    | 0     | 101   | B             |
| ...               | ...         | ...               | ...         | ...   | ...   | ...   | B             |
| 63                | 50          | 50                | 99          | 50    | 0     | 199   | B             |
| 50+25/<br>4<br>56 | 50/2<br>25  | 50+50/<br>4<br>63 | 100/2<br>50 | 50    | 0     | 200   | B             |

| A                 |            | B                 |            | C                 |             |       |               |
|-------------------|------------|-------------------|------------|-------------------|-------------|-------|---------------|
| p_pri             | p_cpu      | p_pri             | p_cpu      | p_pri             | p_cpu       | Lépés | Futó folyamat |
| 56                | 25         | 63                | 50         | 50                | 1           | 201   | C             |
| ...               | ...        | ...               | ...        | ...               | ...         | ...   | C             |
| 56                | 25         | 63                | 50         | 50                | 100         | 299   | C             |
| 50+13/<br>4<br>53 | 25/2<br>13 | 50+25/<br>4<br>56 | 50/2<br>25 | 50+50/<br>4<br>63 | 100/2<br>50 | 300   | C             |

300.  
53    14    56    25    63    50    301    A

301.

KF=0.5, nice=0

# Ütemezési mintapéldák

Ütemezés Round-Robin  
ütemezéssel együtt

KF=0.8, nice=0

| óraütés   | A folyamat |       | B folyamat |       | C folyamat |       | Átutemezés |          | óraütés   |
|-----------|------------|-------|------------|-------|------------|-------|------------|----------|-----------|
|           | p_pri      | p_cpu | p_pri      | p_cpu | p_pri      | p_cpu | előtt fut  | után fut |           |
| kiindulás | 50         | 0     | 50         | 0     | 50         | 0     |            |          | kiindulás |
| 1         | 50         | 1     | 50         | 0     | 50         | 0     | A          | A        | 1         |
| 2         | 50         | 2     | 50         | 0     | 50         | 0     | A          | A        | 2         |
| 3         | 50         | 3     | 50         | 0     | 50         | 0     | A          | A        | 3         |
|           |            |       |            |       |            |       |            |          |           |
| 9         | 50         | 9     | 50         | 0     | 50         | 0     | A          | A        | 9         |
| 10        | 50         | 10    | 50         | 0     | 50         | 0     | A          | B        | 10        |
| 11        | 50         | 10    | 50         | 1     | 50         | 0     | B          | B        | 11        |
| 12        | 50         | 10    | 50         | 2     | 50         | 0     | B          | B        | 12        |
|           |            |       |            |       |            |       |            |          |           |
| 20        | 50         | 10    | 50         | 10    | 50         | 0     | B          | C        | 20        |
| 21        | 50         | 10    | 50         | 10    | 50         | 1     | C          | C        | 21        |
|           |            |       |            |       |            |       |            |          |           |
| 30        | 50         | 10    | 50         | 10    | 50         | 10    | C          | A        | 30        |
| 31        | 50         | 11    | 50         | 10    | 50         | 10    | A          | A        | 31        |
|           |            |       |            |       |            |       |            |          |           |
| 40        | 50         | 20    | 50         | 10    | 50         | 10    | A          | B        | 40        |
| 41        | 50         | 20    | 50         | 11    | 50         | 10    | B          | B        | 41        |
|           |            |       |            |       |            |       |            |          |           |
| 50        | 50         | 20    | 50         | 20    | 50         | 10    | B          | C        | 50        |
| 51        | 50         | 20    | 50         | 20    | 50         | 11    | C          | C        | 51        |
|           |            |       |            |       |            |       |            |          |           |
| 60        | 50         | 20    | 50         | 20    | 50         | 20    | C          | A        | 60        |
| 61        | 50         | 21    | 50         | 20    | 50         | 20    | A          | A        | 61        |
|           |            |       |            |       |            |       |            |          |           |
| 70        | 50         | 30    | 50         | 20    | 50         | 20    | A          | B        | 70        |
| 71        | 50         | 30    | 50         | 21    | 50         | 20    | B          | B        | 71        |
|           |            |       |            |       |            |       |            |          |           |
| 80        | 50         | 30    | 50         | 30    | 50         | 20    | B          | C        | 80        |
| 81        | 50         | 30    | 50         | 30    | 50         | 21    | C          | C        | 81        |
|           |            |       |            |       |            |       |            |          |           |
| 90        | 50         | 30    | 50         | 30    | 50         | 30    | C          | A        | 90        |
| 91        | 50         | 31    | 50         | 30    | 50         | 30    | A          | A        | 91        |
|           |            |       |            |       |            |       |            |          |           |
| 99        | 50         | 39    | 50         | 30    | 50         | 30    | A          | A        | 99        |
| 100       | 58         | 32    | 56         | 24    | 56         | 24    | A          | B        | 100       |
| 101       | 58         | 32    | 56         | 25    | 56         | 24    | B          | B        | 101       |
| 102       | 58         | 32    | 56         | 26    | 56         | 24    | B          | B        | 102       |



# A tradicionális UNIX ütemezés értékelése

- + Egyszerű és hatékony
- + Általános célú időosztásos rendszerben megfelelő
- + Jól kezeli az interaktív és batch típusú folyamatok keverékét
- + Elkerüli az éhezést
- + Jól támogatja az I/O műveleteket végrehajtó folyamatokat
- Nem skálázódik jól: ha sok a folyamat, akkor sok a számítás
- Nem lehet folyamat (csoportok) számára CPU-t garantálni
- Nem lehet a folyamatokat igényeik szerint eltérően ütemezni
- Nincs garancia a válaszidőre
- A többprocesszoros támogatás nem kidolgozott
- A kernel nem preemptív:
  - Ha egy folyamat sokáig fut kernel módban, akkor feltartja az egész rendszert.
  - **prioritás inverzió**: egy alacsonyabb prioritású folyamat feltart egy nála magasabb prioritású folyamatot.

# Modern UNIX ütemezők szempontjai

- Új ütemezési osztályok
  - speciális alkalmazási igények: multimédia, beágyazott, stb.
  - „fair share”: az erőforrások tervezhető elosztása
  - a kernelek is egyre több saját folyamatot (szálat) futtatnak
  - batch, real-time, kernel és interaktív folyamatok keveréke fut egy időben
  - moduláris ütemező rendszer szükséges: bővíthető keretrendszer
- Kernel megszakíthatóság
  - a tradicionális UNIX kernel nem preemptív (szűk keresztmetszet)
  - a többprocesszoros (többmagos) ütemezéshez elengedhetetlen

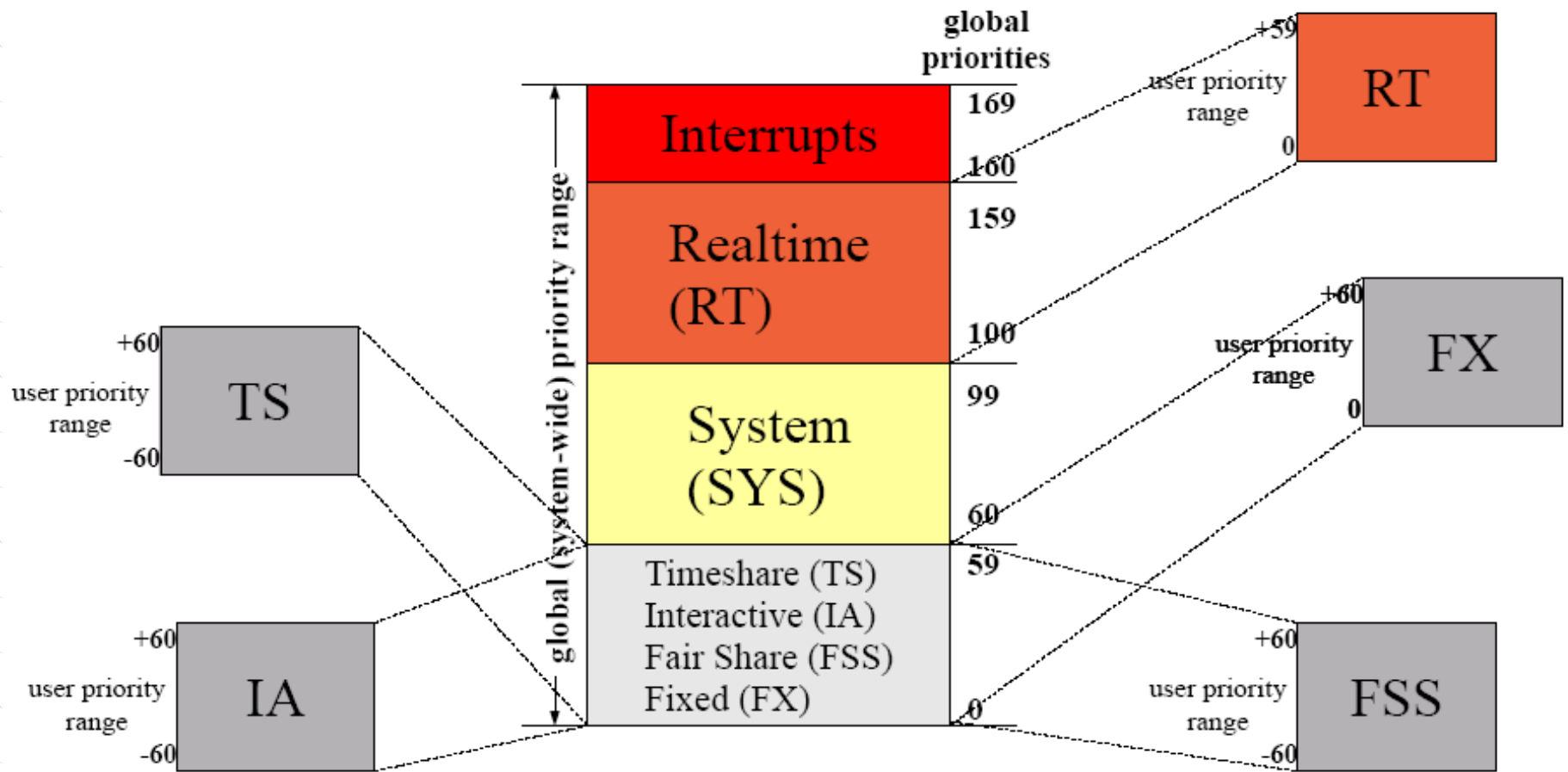
# Modern UNIX ütemezők szempontjai (folyt.)

- Az ütemező erőforrásigénye (teljesítménye)
  - az ütemezés egyre összetettebb feladat (több processzor, többféle sor)
  - nem lenne jó az ütemezésre pazarolni a processzoridőt
  - az ütemezési algoritmus komplexitása lineáris, de inkább  $O(1)$  legyen
- Szálak kezelése (folyamatokat vagy szálakat ütemezünk?)
  - szálak számának növekedésével nő az ütemező terhelése
  - a szálak kerültek előtérbe – ennek megfelelő ütemező kell

# A Solaris ütemező tulajdonságai

- Alapvető tulajdonságok
  - az ütemezés száralapú
  - a kernel is preemptív
  - támogatja a többprocesszoros rendszereket és a virtualizációt (zónák)
- Többféle ütemezési osztály
  - Időosztásos (TS): az ütemezés alapja az, hogy ki mennyit várt/futott
  - Interaktív (IA): azonos a fentivel, de az éppen aktív ablakhoz tartozó folyamat kiemelése
  - Fix prioritás (FX): állandó prioritású folyamatok
  - Fair share (FSS): CPU erőforrások folyamatcsoportokhoz rendelése
  - Real-time (RT): a legkisebb késleltetést biztosító legmagasabb szint
  - Kernel szálak (SYS): az RT szint kivételével mindennél magasabb szint

# A Solaris ütemezési szintjei



# Összefoglalás

- A klasszikus UNIX ütemező
  - felhasználói módban: prioritásos, időosztásos, preemptív
    - többszintű ütemezési tevékenység
    - azonos prioritási szinten Round-Robin ütemezés (időosztás)
    - a prioritás alapja a korábbi CPU használat és a *nice* érték
  - kernel módban
    - statikus prioritás, alapja az ún. alvási prioritás
    - nem preemptív
  - egyszerű, kerüli az éhezést, de rosszul skálázható és nem támogat sok mai alkalmazási igényt (valós idejű rendszerek, multimédia, stb.)
- Modern UNIX ütemezők
  - moduláris felépítésű ütemező rendszer
  - több ütemezési osztály az alkalmazási igényeknek megfelelően
  - többprocesszoros rendszerek támogatása
  - jobb erőforrás-allokáció
  - szálak ütemezése

# CALL-OUT mechanizmus

- Hogy a kernel valamit egy későbbi időpontban hajtson végre
  - Megadás: *timeout* rendszerhívás
  - Törlés: *untimeout* rendszerhívás
- Rendszerkörnyezetben fut → nem éri el a folyamatkörnyezeteket
- Periódikusan ismétlődő (kernel) feladatok végrehajtására jó, pl.
  - Hálózati csomagok küldése,
  - ütemezési és memóriakezelő függvények hívása,
  - készülékek monitorozása,
  - nem IT-vel működő készülékek lekérdezése

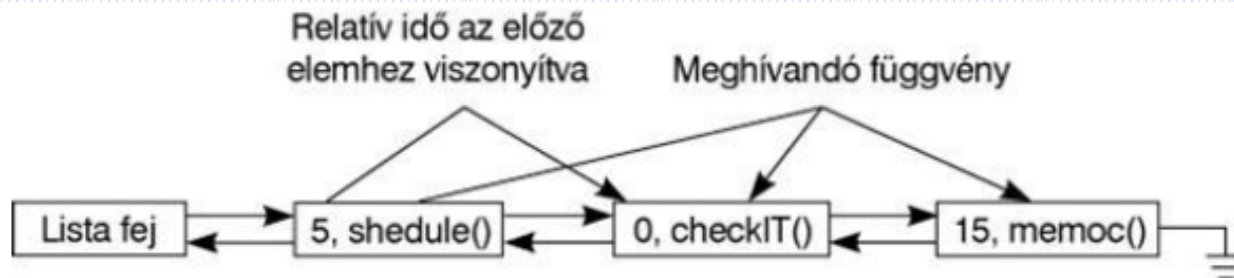
# A call-out-ok meghívása

- Óramegszakítás végzi
- A call-out függvények magas prioritású kernel rutinok, amik maszkolnák az alacsonyabb IT-ket (nem lenne jó), ezért
  - Óramegszakítás → ellenőrzés (itt az ideje egy call-outnak?) → ha igen, akkor jelzőbit beállítása
  - OS: megszakítások kezelése után, minden visszatérés előtt ellenőrzi a bitet → az összes megszakítás kiszolgálása **után**, a **lehető leghamarabb** meghívja a call-out-ot
- A call-out függvények meghívásának adminisztrálása
  - Dinamikus adatszerkezetekben
    - Láncolt listás
    - időkerekés
  - Cél a műveletek végrehajtásának gyorsítása



# Call-out-ok láncolt listában

- Rendezett lista, a call-out-ok „tüzelési” (meghívási) sorrendben következnek egymás után
- Az időt óramegyszakításokban (tikk) számolja
- Relatív időpontokat tárol
- A kernel minden óramegyszakításnál eggyel csökkenti az első listaelem számlálóját. „0”-nál aktivál (lehet több is !)
- Egyszerű kezelés, de hosszú lista lehet



# Call-out-ok időkerékekkel

- A teljes lista (nagyjából egyenlő) részlistákra van bontva
- A részlisták (tüzelési idő szerint) rendezett listák
- Abszolút meghívási időt tárol
- Pl. 10 elemű hashtábla, 1. elem: az időpont 0 mod 10, 2. elem 1 mod 10, ... , 10. elem az időpont 9 mod 10
- Kezelése: óraütés → melyik részlista az aktuális (körbe jár) → idő ↔ tüzelési idő összehasonlítása → ha igen, meghívás, majd a következő elem vizsgálata

